

# Task Card 4 · Build a Team of AI Agents

1 h 45 min hands-on · Claude Code · **Config files, not code — orchestrator + specialists** · For experienced coders

## The goal (the shift you're here to feel)

You will **not** write the code step by step today. You will design a team of agents — their roles, rules, tools and models — and then release a task through it. Your job moves from reviewing every line to engineering the system that produces the lines. It is the same shift as assistant → agent.

## Six concepts the exercise makes concrete

| CONCEPT               | WHAT IT IS                                                                                                                          |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <b>Constitution</b>   | The agent's system prompt — role, principles, behavior. Most of the reliability lives here.                                         |
| <b>Agents</b>         | Scoped workers with narrow jobs. A team of specialists beats one do-everything agent.                                               |
| <b>Tools</b>          | What the agent <i>can</i> do — read, write, run commands.                                                                           |
| <b>Guardrails</b>     | What the agent <i>may</i> do — read-only, denied tools, turn budgets.                                                               |
| <b>Model + effort</b> | The right brain for the job: Haiku for mechanical steps, Sonnet for implementation, Opus + high effort for architecture and review. |
| <b>Orchestration</b>  | How work is delegated. Here: the main Claude Code session is the orchestrator; subagents are the specialists.                       |

## How it works technically (2-minute read)

Subagents in Claude Code are markdown files in `.claude/agents/` in your project folder. YAML frontmatter configures them; the body becomes the system prompt (the constitution):

```
---
name: reviewer
description: Reviews code. Use after every implementation.
tools: Read, Grep, Glob          # guardrail: cannot modify files
model: opus                      # sonnet / opus / haiku / inherit
effort: high                     # low / medium / high
maxTurns: 15                    # guardrail: budget
---
You are a senior code reviewer... (the constitution)
```

The main session delegates automatically based on the description field, or explicitly: *"Use the planner agent to..."*. Four ready-made agents are in the **agents/** folder — copy it to `.claude/agents/` in your project folder.

## Progression (each step teaches one mechanic)

| TIME             | STEP                                                                                                                                    | LESSON                                |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|
| <b>0:00–0:15</b> | Set up: new project folder, copy in <b>agents/</b> , start <code>c laude</code> . Read the four agent files.                            | Anatomy: frontmatter vs constitution. |
| <b>0:15–0:30</b> | Run the task in <b>task/TASK.md</b> <b>without</b> agents first (let Claude do everything in the main thread). Watch the context swell. | Baseline.                             |
| <b>0:30–1:00</b> | Run the same task <b>through the team</b> : have the main session delegate by role. Don't touch the code yourself.                      | Orchestration + delegation.           |

|           |                                                                                                                                               |                                     |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------|
| 1:00–1:20 | <b>Try to break a guardrail.</b> Ask the reviewer to "fix the bug it found". It should refuse/fail (read-only). Discuss why that's a feature. | Guardrails vs instructions.         |
| 1:20–1:35 | Modify: swap model/effort on one agent (e.g. reviewer to haiku/low) and re-run the review. Compare quality.                                   | Model choice is a design parameter. |
| 1:35–1:45 | Lightning demo: show your agent files + a delegation in the transcript.                                                                       | —                                   |

#### DONE WHEN

You can show (1) your agent definitions, (2) a run where the main session delegated to at least two agents, and (3) a finished result without you steering the code manually.

## Wrap-up discussion prompts

- How much of the reliability came from the constitution vs from the guardrails?
- When is a team of narrow agents better than one capable agent — and when is it just overhead?
- What in your normal review-driven workflow would you dare delegate like this tomorrow?

#### IF YOU GET STUCK

Come to the AI table. Most common: agents not found (check the folder is exactly `.claude/agents/` in the project root; restart `claude` if the folder was created after the session started).